Chupacabras AR

Desarrollo, tecnologías y funcionamiento

Chupacabras AR es una aplicación Android que sitúa personajes tridimensionales animados sobre un marcador impreso. Al abrir la cámara y enfocar el AprilTag, aparecen el chupacabras y una oveja; el usuario puede observar la escena desde distintos ángulos moviendo el teléfono. El desarrollo que aquí se presenta comprende dos fases: la figura con respiración y su evolución hacia una secuencia de ataque.

La experiencia incluye pantalla de inicio, solicitud de permiso de cámara, acceso a privacidad y avisos, seguimiento del marcador y animación sincronizada. La escena se oculta cuando el marcador deja de detectarse y reinicia al recuperarlo. También reinicia al volver de segundo plano o comenzar otra sesión desde Inicio. El posicionamiento se mantiene relativo al tag mientras este es detectado.

El chupacabras se construyó en Blender a partir de referencias visuales. Se combinaron volúmenes anatómicos y formas tubulares, aplicando remallado, suavizado y reducción de geometría. La revisión Nocturno R08 refinó cráneo, hocico, órbitas, dentadura, orejas y cresta. La piel se creó mediante materiales procedurales, con pigmentación, poros y pliegues, y se preparó un atlas de texturas de 2048 × 2048 píxeles. Las referencias visuales guiaron la forma; las texturas se generaron dentro del proceso de producción.

La primera fase culminó al colocar ese modelo sobre el marcador y añadirle un ciclo de respiración de cuatro segundos. Un esqueleto inicial de cuatro huesos permitía expandir el tórax y acompañar el movimiento con cabeza y cola, conservando el apoyo de las patas. El modelo, las texturas y la animación se integraron en Unity y se validaron en el Moto G20.

En la segunda fase se añadió la oveja del *Farm Animal Pack*, de Quaternius (2018), distribuido con licencia CC0 1.0. Se adaptaron colores, escala y pesos del esqueleto original; una subdivisión suavizó su silueta. El rig del chupacabras se amplió a 17 huesos para articular patas y mandíbula, conservando su geometría y sus texturas. La oveja mantiene 24 huesos. El conjunto suma 112.822 triángulos.

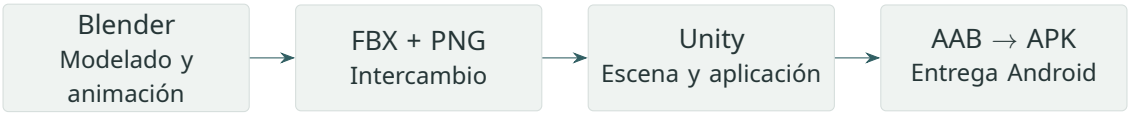
La actuación final comprende pastoreo, anticipación, salto con desplazamiento, mordida y forcejeo sin sangre, liberación y regreso caminando. Se corrigieron el alcance de las patas, el contacto, la deformación de la cola y la continuidad del recorrido. A los 13,5 segundos el chupacabras entra en respiración continua de cuatro segundos y la oveja queda inmóvil. El cierre conserva al chupacabras sobre el marcador.

Tecnologías y procesos de producción

Las herramientas principales son Blender 5.2.2 LTS, para crear los recursos, y Unity 6.3 LTS, versión 6000.3.22f1, para integrarlos y ejecutar la aplicación. En Unity se utiliza C# para la lógica y Universal Render Pipeline (URP) 17.3.0 para iluminación y materiales. Los materiales PBR describen color, relieve y respuesta de la superficie a la luz.

Python es el lenguaje de automatización de Blender. Los scripts usan su API: `bpy`, `bmesh` y `mathutils`, junto con NumPy, para trabajar con geometría, materiales, transformaciones y datos numéricos. Generan recursos y verificaciones reproducibles; Python participa en la producción, mientras que la aplicación Android ejecuta el resultado preparado.

El seguimiento utiliza `jp.keijiro.apriltag` 1.0.3, de Keijiro Takahashi, que conecta AprilTag con Unity. Reconoce el marcador y entrega posición y rotación para ubicar los personajes. Su función es comparable a la detección y posicionamiento de objetivos de Vuforia, cuyo SDK tiene un alcance más amplio. La integración utilizada se apoya en bibliotecas nativas y en Unity Jobs, Burst 1.8.30 y Mathematics 1.3.2.



La exportación FBX pertenece al flujo de intercambio de Blender y Unity. El archivo lleva mallas, esqueletos y animación horneada; las texturas se entregan como PNG y los JSON registran datos de verificación. Hornear significa guardar las poses calculadas: el móvil reproduce esas curvas sin resolver IK ni simular físicamente el ataque. Unity importa el rig como Generic y utiliza un Animator compartido para sincronizar ambos personajes. El ancla del marcador queda fuera de las curvas de animación.

La compilación Android de Unity utiliza IL2CPP y ARM64 para producir el AAB firmado. Después, bundletool genera la APK universal instalable a partir de ese AAB. FBX y AAB son formatos de producción y entrega; bundletool es una herramienta del ecosistema Android. Como apoyo se usaron ADB para instalar y revisar la app, FFmpeg para videos de revisión y Git con Git LFS para versionar fuentes y archivos grandes. Los comandos se automatizaron con Bash.

Funcionamiento y estado final

Al pulsar «Abrir cámara», la app solicita permiso y obtiene imágenes mediante WebCamTexture, una API de Unity. Corrige giro y reflejo, procesa el AprilTag de familia `tagStandard41h12`, ID 0, y coloca el conjunto en su pose. El lado de detección configurado mide 10 cm. El modelo y sus luces dependen de esa ancla; un Animator reproduce el ataque una vez y después el reposo en bucle. La interfaz usa IMGUI, también integrado en Unity.

La versión final documentada es la **0.2.8 / 208**, entregada el 11 de octubre de 2026 y aceptada en Moto G20 con Android 11. Su envolvente animada es de aproximadamente 80,2 × 91,6 × 35,1 cm virtuales. El perfil de 180,70 segundos registró **29,33 fps de media**. Se verificaron seguimiento, reproducción, geometría, recuperación de cámara y empaquetado; la entrega contiene AAB firmado y APK derivada. Requiere Android 8.0 o superior y ARM64. No se ha publicado en Google Play. La escala física exacta depende de la calibración existente; siguen pendientes pruebas prolongadas, S23 y ejecución en equipos con páginas de memoria de 16 KB.

Referencias

Android Developers. (s. f.-a). *Android Debug Bridge (adb)*. https://developer.android.com/tools/adb

Android Developers. (s. f.-b). *bundletool*. https://developer.android.com/tools/bundletool

AprilRobotics. (s. f.). *AprilTag* [Software]. GitHub. https://github.com/AprilRobotics/apriltag

Blender Foundation. (s. f.-a). *Blender* [Software]. https://www.blender.org/

Blender Foundation. (s. f.-b). *Blender Python API*. https://docs.blender.org/api/current/index.html

Blender Foundation. (s. f.-c). *FBX*. Blender 4.5 Manual. https://docs.blender.org/manual/en/4.5/addons/import_export/scene_fbx.html

FFmpeg Developers. (s. f.). *FFmpeg* [Software]. https://ffmpeg.org/

Git Project. (s. f.). *Git* [Software]. https://git-scm.com/

GitHub. (s. f.). *Git Large File Storage* [Software]. https://git-lfs.com/

GNU Project. (s. f.). *Bash* [Software]. https://www.gnu.org/software/bash/

Microsoft. (s. f.). *Documentación de C#*. https://learn.microsoft.com/es-es/dotnet/csharp/

NumPy Developers. (s. f.). *NumPy documentation*. https://numpy.org/doc/stable/

PTC. (s. f.). *Vuforia features*. Vuforia Engine Library. https://developer.vuforia.com/library/vuforia-engine/getting-started/vuforia-features/

Python Software Foundation. (s. f.). *Python* [Lenguaje de programación]. https://www.python.org/

Quaternius. (2018, junio). *Farm Animal Pack* [Modelos 3D]. https://quaternius.com/packs/farmanimal.html

Takahashi, K. (s. f.). *jp.keijiro.apriltag* (versión 1.0.3) [Software]. GitHub. https://github.com/keijiro/jp.keijiro.apriltag

Unity Technologies. (s. f.). *Unity 6.3 documentation*. https://docs.unity3d.com/6000.3/Documentation/Manual/

Las versiones y los resultados del proyecto proceden de sus fuentes y registros locales: `docs/rediseno_nocturno.md`, `docs/asset_oveja.md`, `docs/ataque_natural_1m.md` y la configuración de la copia Unity `google_play`. Los enlaces públicos documentan las herramientas y la procedencia de los recursos.